fastPLS: a memory-aware accelerated SIMPLS implementation for high-dimensional biomedical data with optional CUDA and Metal routes

Dupe Ojo[a,†], Alessia Vignoli[b,c,†], Stefano Cacciatore[a,d,*], Leonardo Tenori[b,c,*]

[a] Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology, Cape Town 7925, Western Cape, South Africa

[b] Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy

[c] Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy

[d] Department of Integrative Biomedical Sciences, Institute of Infectious Disease and Molecular Medicine, University of Cape Town, Cape Town 7925, South Africa

†These authors contributed equally and share first authorship.

*Co-corresponding authors: Stefano Cacciatore, stefano.cacciatore@icgeb.org; Leonardo Tenori, tenori@cerm.unifi.it.

Abstract

Background and objective: High-dimensional biomedical partial least-squares (PLS) workflows, including multivariate NMR prediction and repeated validation, can be limited by the sequential cost and storage of SIMPLS, de Jong's sequential PLS algorithm. We developed fastPLS to reduce this burden while retaining the component equations.

Methods: Central evidence used archived fastPLS 0.99.25 (Git commit 7887401b09e2). The implementation combines compiled incremental SIMPLS updates, reuse of sequential deflation quantities, compact latent prediction, and optional implicit cross-covariance products. We

evaluated SIMPLS and PLS-SVD, which derives components from a singular value decomposition of the predictor-response cross-covariance. Classification used maximum-score (argmax) decoding or linear discriminant analysis. Fixed-control implicitly restarted Lanczos bidiagonalization (IRLBA) provided the deterministic numerical reference; approximate randomized SVD (rSVD) and accelerator routes were evaluated separately under predefined criteria.

Results: An independent dense-LAPACK de Jong panel completed all 82 component-prefix comparisons without numerical failure. Outside an intentionally near-tied singular-value case, maximum held-out prediction error was 4.53 x 10^-15; the tied case retained a 0.015-degree maximum subspace angle and 3.36 x 10^-4 relative prediction error. Against pls::simpls.fit, fastPLS was faster on five of nine datasets in repeated ordinary public-workflow comparisons, with identical accuracy and a maximum 4.85-fold speed-up. Across the broader R-package panel, fastPLS was the fastest tested workflow on seven of nine classification datasets. fastPLS also completed the 13,000 by 28,355-response NMR benchmark and the 1,000,000-row ImageNet/DINOv2 stress test; no tested external R workflow completed ImageNet, and only limited external routes were feasible for NMR. At 50 NMR SIMPLS components, CPU IRLBA, CPU rSVD, and CUDA rSVD required 692.21, 152.09, and 3.91 s, respectively, with RMSD 0.0007561, 0.0007560, and 0.0007561. In a separate float32 comparison, IKPLS matched the ImageNet argmax endpoint but could not fit the selected NMR model because its retained 50-component coefficient tensor alone required 68.66 GiB.

Conclusions: fastPLS reduces the computational barrier to sequential SIMPLS in large biomedical workflows while making solver and backend qualification explicit.

Keywords: partial least squares; accelerated SIMPLS; randomized SVD; CUDA; Metal; metabolomics

1. Introduction

Partial least squares (PLS) combines supervised dimension reduction with prediction and is widely used when biomedical predictors are correlated, high dimensional, or accompanied by multivariate responses [1-3]. Applications include cancer metabolomics and prediction of multiple nuclear magnetic resonance (NMR) spectra from one acquisition [4-7].

NMR spectrum prediction illustrates a particularly demanding multivariate setting. In the previous scientific workflow that motivated this work, 1,200 spectra represented by 13,000 NOESY predictor bins were used to predict 28,355 diffusion-edited spectral intensities [7]. Unlike prediction of a single clinical endpoint, this task requires fitting and evaluating tens of thousands of correlated responses simultaneously. The predictor-response cross-covariance alone contains approximately 369 million values, or 2.75 GiB in float64, before latent factors, coefficients, predictions, and validation workspaces are considered. Repeated component selection further multiplies this cost, making both execution time and memory central methodological constraints.

The same computational pressure arises from increasing spectral resolution, single-cell sample counts, foundation-model embeddings, and repeated validation. KODAMA, for example, repeatedly fits cross-validated PLS discriminant models [8,9]. In medical imaging, foundation models such as UNI and Prov-GigaPath generate large dense tile or slide representations for downstream supervised analysis [31,32]. Historical ImageNet/DINOv2 embeddings were

therefore retained only as a partially reproducible engineering proxy for this post-extraction matrix regime, not as biomedical validation [28,29].

Several PLS formulations and software implementations address different parts of this problem. SIMPLS constructs sequential components without explicitly deflating the predictor matrix [11]; PLS-SVD provides a one-shot cross-covariance formulation [10]; and OPLS and kernel PLS extend the framework to orthogonal filtering and nonlinear relations [12-14]. Established R packages provide widely used reference and application workflows, while accelerated approaches use iterative or randomized low-rank solvers, implicit products, compiled linear algebra, or accelerator execution [15,16]. The IKPLS software implements Improved Kernel PLS with NumPy and JAX [33-35]; because it is not de Jong SIMPLS, we evaluate it as a separate end-to-end software comparator rather than as evidence of estimator equivalence.

We present fastPLS, whose principal methodological contribution is an accelerated compiled execution of de Jong SIMPLS. Sequential deflation, orthogonalization, and component definitions are retained, while rank-one deflation products and previously computed sequential quantities are reused, coefficient and fitted-value updates are accumulated incrementally, and compact latent prediction avoids unnecessary dense coefficient and prediction paths. Optional caching and implicit predictor-response products provide secondary workload-specific memory optimizations. The R interface also provides PLS-SVD, OPLS, kernel PLS, compiled single and nested validation, and explicit solver and backend diagnostics. Low-rank solvers, float32 execution, linear discriminant analysis, and CPU, NVIDIA CUDA, and Apple Metal routes support the central SIMPLS implementation. The benchmark therefore separates numerical validation against pls::simpls.fit, comparison with available R PLS workflows, and a distinct

cross-language comparison with IKPLS. The GPL-3 R package uses reusable components from the MIT-licensed kodama-cpp project.

2. Materials and methods

2.1 Software architecture

The public model-fitting interface selects the PLS family, component count, low-rank solver, execution backend, and classification head. Single and nested cross-validation use the same dispatch contract. Unsupported combinations stop before fitting, requested estimators are not silently replaced, and fitted objects record solver diagnostics and host/device residency (Supplementary Tables S1 and S9). Complete API, deprecation, and compatibility details are provided in the package documentation and Supplementary Section S1.

Let a index a component. A denotes the maximum retained component count, C the requested component-count set, and K denotes the number of cross-validation folds. Lowercase k is reserved for retrieval cutoffs.

The architecture separates preprocessing, the PLS estimator, low-rank direction extraction, and CPU, NVIDIA CUDA, or Apple Metal execution (Figure 1). Benchmark rows record both the requested and executed estimators, and any mismatch is treated as a failed run.

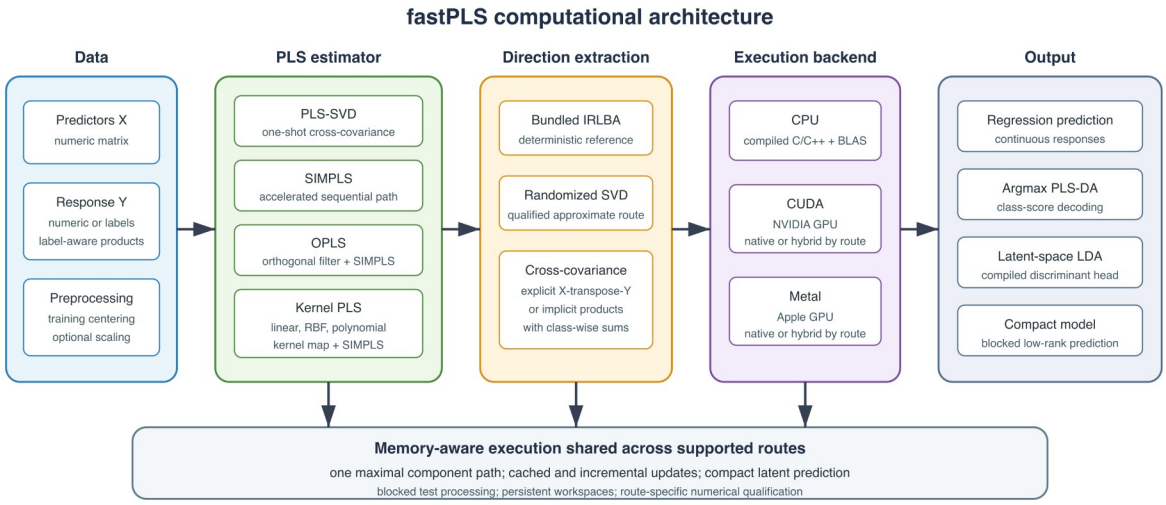

Figure 1. fastPLS architecture separating response representation, PLS estimator, low-rank solver, backend, and prediction head. Accelerator routes may be native or explicitly reported as hybrid.

CPU routines use compiled C/C++ and R-linked BLAS/LAPACK; CUDA uses NVIDIA CUDA/cuBLAS and Metal uses Apple Metal Performance Shaders. The external comparison used one effective BLAS thread, so no multicore speed-up is claimed. Versions, build flags, residency, and thread settings are in Supplementary Tables S1 and S4a.

2.2 Accelerated SIMPLS execution and related PLS models

Predictors were centred and optionally scaled using training statistics; numeric responses were centred. Classification used a multivariate PLS-DA response, with class-wise products replacing dense one-hot matrices on large-class routes.

The accelerated method targets SIMPLS; PLS-SVD is retained as a one-shot comparator. Both operate on the centred cross-covariance

$$S = X^T Y .$$

PLS-SVD decomposes the centred cross-covariance once at the largest valid requested rank and reuses prefixes. Its component count is bounded by cross-covariance rank and, for C centred classes, by C-1.

SIMPLS follows de Jong's sequential score, loading, orthogonalization, and rank-one deflation equations [11]. As in pls::simpls.fit, one fit supplies the standard sequential path through all components up to the requested maximum; fastPLS does not claim this path construction as novel. Its main computational contribution is the incremental reuse of work along that path: rank-one deflation products are reused, coefficient and fitted-value quantities are accumulated as each component is added, and compact prediction retains latent factors instead of dense coefficient and prediction paths. Caching X-transpose-X when its setup can be amortized and using implicit products when the predictor-response cross-covariance is limiting are secondary workload-specific optimizations. Every component invokes a fresh rank-one direction calculation on the current deflated state. Candidate-direction blocks, cross-component warm starts, and adaptive refresh were rejected and are not used by the public CPU, CUDA, or Metal paths.

$$S_{a+1} = S_a - v_a \left(v_a^T S_a \right).$$

The accelerated path reuses sequential deflation quantities and updates prediction-related quantities incrementally. Compact latent factors and blocked prediction avoid retaining dense

coefficient or prediction paths. X-transpose-X caching, optional implicit products, and rSVD affect the computational route or storage, not the SIMPLS deflation equations.

Algorithm 1 summarizes the unchanged estimator and the retained execution updates. The minimally optimized compiled baseline, asymptotic costs, storage terms, and detailed de Jong mapping are provided in Supplementary Table S2 and Section S12.1.

Algorithm 1. Accelerated incremental SIMPLS execution. Direction extraction uses deterministic IRLBA or approximate rSVD; score construction, orthogonalization, and deflation follow de Jong [11].

Input: centred X; numeric, indicator, or label-aware response Y; maximum component count A; requested component-count set C; solver.
1. Define $S_0 = X^TY$ explicitly, or define operator products $S(v) = X^T(Yv)$ and $S^T(u) = Y^T(Xu)$; label-aware products use class sums.
2. If $p \leq n$ and storage permits, cache $G = X^TX$. Initialize R, Q, and V as empty; set fitted values $\hat{Y}_0 = 0$.
3. For component a = 1, …, A:
3.1 Extract the leading left singular direction r_a in predictor space from state S_{a-1} using a fresh IRLBA solve or a fresh oversampled rSVD sketch.
3.2 Set $t_a = Xr_a$; divide r_a and t_a by $\|t_a\|_2$.
3.3 Compute predictor and response loadings $p_a = X^Tt_a$ and $q_a = Y^Tt_a$.
3.4 Orthogonalize $v_a = (I - VV^T)p_a$ and normalize v_a.
3.5 Deflate $S_a = S_{a-1} - v_a(v_a^TS_{a-1})$, or update the equivalent implicit operator.
3.6 Append r_a, q_a, and v_a; update $B_a = B_{a-1} + r_aq_a^T$ and $\hat{Y}_a = \hat{Y}_{a-1} + t_aq_a^T$.
3.7 If a ∈ C, store only the requested coefficient/prediction snapshot or compact latent representation.
Output: the standard sequential component path; requested outputs are retained as dense snapshots or compact latent factors.

OPLS and kernel PLS are secondary extensions that reuse the accelerated SIMPLS core. OPLS first applies an orthogonal predictor filter [12]. Linear kernel PLS dispatches to the linear route; RBF and polynomial kernels form an n-by-n Gram matrix [13,14], restricting nonlinear models to moderate sample sizes.

2.3 Truncated SVD and memory-aware computation

Direction extraction is modular rather than the principal estimator contribution. float64 CPU fitting supports fixed-control IRLBA [15] as the deterministic numerical reference and approximate rSVD [16], which uses a Gaussian range sketch, orthonormalization, power iterations, and a reduced decomposition. IRLBA is an iterative truncated solver, not an exact dense SVD. Archived release 0.99.25 uses rSVD oversampling 20 and two power iterations for the evaluated CPU/CUDA routes. Explicit unqualified overrides warn and are recorded in model diagnostics. Controls, seeds, validation thresholds, and route-specific status are consolidated in Supplementary Sections S13-S14 and Tables S8-S9.

For selected large shapes, fastPLS avoids materializing S and evaluates the same operator through

$$S\Omega = X^{\top}(Y\Omega), S^{\top}Q = Y^{\top}(XQ).$$

When explicit cross-covariance is large, identical centred/scaled operators are evaluated as X-transpose-(YV) and Y-transpose-(XU). Class-wise sums avoid dense indicator responses. Blocking changes storage, not the global operator.

Compact latent prediction processes large test matrices in row blocks and releases temporary scores after each block. For the ImageNet top-5 analysis, predictions were similarly blocked and classification counts were accumulated online, avoiding a full 281,167-by-1,000 double score matrix.

2.4 CPU, CUDA, Metal, and precision

Standard R matrices use eight-byte float64 values; float-package inputs use four-byte float32 values. float64 is the reference. Precision support and backend residency are route specific; the authoritative capability classifications are in Supplementary Tables S1 and S9. On Windows, a portable float-package CPU fallback supports rSVD PLS-SVD, SIMPLS, and linear-kernel SIMPLS with argmax, whereas native compiled float32 OPLS, nonlinear kernel PLS, and LDA remain unavailable.

2.5 Prediction, classifiers, and validation

For regression analyses, the fitted models produced continuous response estimates, whereas classification was performed by either maximum-score PLS-DA decoding or linear discriminant analysis (LDA) of the PLS scores. The LDA model used the pooled within-class covariance, class priors, and Cholesky-based linear solves, with deterministic trace-scaled regularization introduced only when factorization failed [17,18]. For imbalanced classification, model selection could instead maximize balanced accuracy, defined as the unweighted mean of class-specific

recalls, and nested permutation testing evaluated the same selected endpoint rather than substituting dummy-response Q^2. Permutation inference applied the finite Monte Carlo correction (b+1)/(B+1), where b denotes the number of successful null statistics at least as extreme as the observed statistic and B the number of successful null fits. Independent observations were permuted by row, whereas repeated observations identified by the grouping constraint were exchanged as complete blocks within equal-size exchangeability strata. This procedure preserved group sizes, within-group responses, and class frequencies while holding fold assignments and randomized-solver seeds fixed across observed and null analyses. Failed null fits were excluded from B and reported explicitly.

$$\Sigma = \{T^\top T - \sum_c n_c \mu_c \mu_c^\top\} / max(1, n - C).$$

$$\delta_c(t) = t^\top w_c - \frac{1}{2}\mu_c^\top w_c + \log(n_c / n).$$

Fold construction, fitting, prediction, and metric accumulation remain compiled where supported; grouped observations can be constrained to one fold. Classification tuning uses held-out accuracy by default or balanced accuracy when equal weighting of class recalls is required. Regression tuning uses held-out RMSD by default and can instead use observed-mean cross-validated R^2 or fold-training-mean cross-validated Q^2. Full-data training R^2 is returned only as a descriptive fit statistic and is never a component-selection endpoint. Independent-test Q^2 uses the complete training-response mean. For PLS-DA, training R^2 and cross-validated Q^2 are calculated on dummy-coded responses and are distinct from decoded-label accuracy and balanced accuracy. Exact formulas and denominator conventions are given in Supplementary Section S4.

2.6 Benchmark and reproducibility design

The broader benchmark design covered biomedical and computational tasks spanning metabolomics, NMR, CITE-seq, tissue and cancer omics, single-cell transcriptomics, drug response, and image embeddings [7,20-30]. Dataset provenance, acquisition or access requirements, construction of the prepared analysis matrices, preprocessing, response encoding, dimensions, split units and seeds, and component grids are documented for every dataset in Supplementary Section S5 and Table S3. Redistribution restrictions and executable acquisition instructions are provided in Supplementary Section S5.6 and the repository file benchmark/DATA_ACQUISITION.md. Archived central evidence comprises deterministic SIMPLS validation, external comparison, controlled scaling and solver qualification, selected CPU/CUDA routes, and the NMR case study. Methods used identical stored splits and training-only component grids within each analysis. Cross-dataset selected points are treated as fixed computational workloads when their grids were boundary or rank constrained. Runtime included fitting and prediction; memory definitions, endpoint definitions, and uncertainty limitations are detailed in Supplementary Sections S4 and S7.

A separate historical ImageNet/DINOv2 analysis used 1,281,167 stored 1,024-dimensional embeddings. The exact DINOv2 checkpoint, pooling rule, extraction script, and auditable image-to-row mapping were not retained; the 1,000,000/281,167 split was noncanonical and had informed earlier component choices. Only the downstream PLS fitting and prediction stages were rerun with archived fastPLS 0.99.25. Consequently, this analysis is partially reproducible and supports only matrix-processing feasibility, not representation-level, biomedical, or comparative predictive claims.

Five-fold training-only selection was performed separately by PLS family. Of 46 evaluated family-dataset choices, 24 were at a tested-grid boundary and nine were limited by response rank; these 33 choices are reported as constrained, not optimal. They define reproducible benchmark workloads and do not support unconstrained family-level predictive claims. The central NMR analysis used an expanded 1-300-component training-only grid and a one-standard-error rule, yielding interior family-specific settings. Accelerator speed-up was interpreted only for numerically concordant paired predictions.

A controlled one-factor-at-a-time SIMPLS study varied training observations (500-10,000), predictors (50-2,000), responses (10-1,000), retained components (2-80), requested prefixes (1-20), exact cross-covariance rank (5-80), classes (5-200), and explicit cross-covariance storage (2-64 MiB). Each point used three matched seeds, a deterministic float64 CPU-IRLBA reference, and rSVD with oversampling 20 and two power iterations. Fresh processes recorded fit and prediction time, baseline-corrected peak host RSS, CUDA memory, and numerical agreement; speed crossovers were inferred only when every replicate met tolerance.

Estimator preservation was first assessed against an independent de Jong implementation using dense LAPACK SVD on fixed well-conditioned, nearly tied, rank-deficient, collinear, $p < n$, $p > n$, high-response, effective-rank-boundary, regression, and dummy-response classification cases. IRLBA was then assessed separately as a deterministic iterative numerical reference against pls::simpls.fit. Approximate rSVD was evaluated in a third, separate qualification using prediction, subspace, decoded-label, and endpoint screens over seeds 1, 7, 19, 43, and 123. Exact definitions and results are in Supplementary Sections S12-S14 and Tables S7-S9.

To distinguish numerical computation from the cost of constructing ordinary model objects, external timing was assessed under two predefined output profiles. The minimum-output profile

compared deterministic float64 SIMPLS while requiring the complete coefficient path, centering quantities, and final held-out predictions; scores, loadings, fitted-value arrays, and variance summaries were suppressed, and pls::simpls.fit was called with stripped = TRUE. The public-workflow profile retained each implementation's ordinary model object together with the final predictions. The strict paired comparison comprised nine classification datasets, two output profiles, fastPLS and pls::simpls.fit, and three fresh-process repetitions, yielding 108 planned runs. A broader workflow panel compared fastPLS argmax and LDA with the available classification workflows from independent R PLS packages on the same nine datasets; package-specific model objects and unsupported configurations were retained and interpreted as an end-to-end software comparison. Package and data loading were completed before timing, no numerical warm-up was applied, and each repetition used one effective BLAS thread and a common 10,000-s timeout. The analysis retained median time, interquartile range, completed repetitions, failures, model-object size, baseline and peak process resident-set size, and final held-out predictions.

2.7 Cross-language software comparison

Numerical equivalence and software-level performance were assessed separately. Deterministic float64 fastPLS SIMPLS was first compared with the de Jong SIMPLS implementation in pls::simpls.fit to evaluate the numerical kernel. A separate archived-release CPU experiment compared fastPLS 0.99.25 with the two NumPy formulations provided by IKPLS 6.1.2. The score-explicit formulation computes and retains training scores, whereas the cross-product formulation fits from the predictor and predictor-response cross-products without retaining training scores. The repeated float64 comparison used Breast, MetRef, and CIFAR-100 with 10, 22, and 50 components, respectively. Both IKPLS formulations, deterministic fastPLS IRLBA,

and approximate fastPLS rSVD were executed three times for each dataset, giving 36 planned runs. All routes used identical stored splits, externally training-centred predictors, centred one-hot responses, component counts, final held-out prediction, fresh processes, and one effective thread. A separate float32 feasibility extension evaluated public IKPLS 6.1.2 on NMR and ImageNet. NMR was attempted at one and five components under a 10-GiB virtual-memory guard; the coefficient-path storage requirement at 50 components was calculated analytically. ImageNet was evaluated at 100, 200, 500, and 1,000 components using pre-centred float32 arrays, blocked held-out prediction, and one CPU thread; float32 centring and one-hot-response construction were timed separately from model fitting. Because IKPLS and SIMPLS implement different estimators and retain different internal state, both experiments were interpreted as end-to-end software or feasibility comparisons rather than estimator-matched benchmarks.

2.8 Large-scale case-study protocols

The NMR case study comprised 1,200 training and 321 held-out spectra, with 13,000 predictors and 28,355 response intensities. The 4.6-4.8 ppm predictor interval was defined a priori from the chemical-shift labels and set to zero in both training and test predictors before any training-only split; no response coordinate was masked and no held-out response information informed preprocessing. Component selection used five paired training-only splits, family-specific grids spanning 1-300 components, and the one-standard-error rule. Family-selected implementation comparisons held the split, precision, response target, and component count fixed. Approximate rSVD used oversampling 20, two power iterations, and seed 123. A separate implementation-only analysis fixed both families at 100 components, and the deposited 165-component PLS-SVD/IRLBA workflow was treated only as historical scientific context.

The historical ImageNet/DINOv2 feasibility analysis used 1,281,167 stored 1,024-dimensional embeddings, partitioned noncanonically into 1,000,000 training and 281,167 held-out rows. The exact DINOv2 checkpoint, pooling rule, feature-extraction script, and independently auditable image-to-row mapping were unavailable. Downstream label-aware float32 CUDA-rSVD SIMPLS fitting and blocked prediction were rerun with fastPLS 0.99.25. Argmax and LDA were evaluated at the shared component prefixes 100, 200, 300, 400, 500, 600, 700, 800, 900, and 1,000; one maximal fit supplied every prefix. rSVD used oversampling 20, two power iterations, and seed 123. Because the split had informed earlier component choices and each configuration was measured once, this analysis was predefined as a partially reproducible engineering stress test rather than a comparative or biomedical predictive evaluation.

3. Results

Results are organized around the accelerated incremental SIMPLS execution and its memory-saving prediction path. We first compare complete single-CPU classification workflows across independent R implementations, then separate numerically concordant CPU, CUDA, and Metal execution. NMR provides the principal biomedical high-response case study, while ImageNet is retained as a qualified foundation-model-scale feasibility analysis. Exact estimator validation, approximate-solver qualification, route diagnostics, and complete result tables remain in the Supplementary Material.

The compiled dense-LAPACK SIMPLS path completed 82/82 component-prefix comparisons with the independent exact-reference updates and no convergence failure. Median and 95th-percentile relative held-out prediction errors were 7.62 x 10^-16 and 2.72 x 10^-15. All non-tied cases remained at floating-point precision; the deliberately near-tied case illustrated non-identifiability of individual singular vectors but retained close predictive and subspace

agreement. Separately, IRLBA SIMPLS met the predefined tolerances in all 117 comparisons with pls::simpls.fit. rSVD was not used as exact estimator-preservation evidence.

3.1 Repeated comparison with an independent SIMPLS implementation

All 108 planned paired runs completed, and held-out accuracy was identical for every fastPLS-pls::simpls.fit pair. With minimum common prediction outputs, fastPLS was faster on four datasets and pls::simpls.fit on five; the largest fastPLS advantage was 1.48-fold on GTEx v8. Under ordinary public workflows, fastPLS was faster than pls::simpls.fit on five datasets, including 2.39-fold on CIFAR-100, 3.35-fold on Retina, and 4.85-fold on Tabula Muris. Corresponding exact held-out counts were 8,739/10,000, 21,684/22,406, and 40,077/50,059. These are conditional row-level computational endpoints; because Retina and Tabula Muris rows are cells rather than independent biological replicates, binomial intervals are not interpreted as biological generalization intervals. In the broader R-package panel, fastPLS had the lowest observed total time on seven of nine classification datasets. The exceptions were TCGA-BRCA and TCGA-HNSC, where plsgenomics was faster (Figure 2; Supplementary Figure S3 and Tables S10a-S10e).

The magnitude of the observed workflow gains was influenced by differences in output retention. On CIFAR-100, the compact fastPLS fit object occupied 1.39 MiB, compared with 7,777.99 MiB for the ordinary pls::simpls.fit object. Median complete-process peak resident-set size was 2,493.57 and 13,415.25 MiB, respectively; after correction for comparable pre-fit baselines of 1,906.80 and 1,906.62 MiB, the corresponding peak increments were 586.77 and 11,508.84 MiB. The largest theoretically retained dense object was the 0.586-MB final coefficient matrix for fastPLS and a 3,814.70-MB fitted or residual response path for pls::simpls.fit. These measurements characterize the specified complete workflows rather than

isolated algorithmic workspaces. When both implementations retained complete coefficient paths, the fit objects occupied 59.30 and 58.60 MiB, the corrected increments were 69.22 and 127.13 MiB, and the speed-up was 1.21-fold. The broader package panel therefore remains a workflow comparison involving implementation-specific outputs (Supplementary Tables S10c-S10e).

3.2 Cross-language software comparison

The repeated float64 CPU comparison on Breast, MetRef, and CIFAR-100 completed all 36 planned runs. The IKPLS cross-product formulation was fastest, with median total times of 0.00059 s on Breast, 0.00298 s on MetRef, and 0.218 s on CIFAR-100, compared with 0.005, 0.033, and 3.585 s for fastPLS rSVD. Breast accuracy was identical (94.29%). IKPLS reached 75.0% on MetRef and 70.95% on CIFAR-100; fastPLS rSVD reached 77.0% and 72.13%, while deterministic fastPLS IRLBA reached 77.0% and 70.77%. On CIFAR-100, complete-process peak resident-set size was 584 MiB for the IKPLS cross-product formulation and 1,178 MiB for fastPLS rSVD, including their different language runtimes and memory allocators. In the separate single-run float32 extension, IKPLS completed all four ImageNet settings. At 1,000 components it reached 79.99% top-1 and 95.61% top-5 accuracy, required 197.06 s for fitting plus blocked prediction, and reached 11.79 GiB peak process RSS; the separately measured float32 centring and one-hot preparation required 26.36 s. Accuracy closely matched fastPLS argmax at the same prefix (79.98% and 95.61%), but runtime is not estimator matched because fastPLS used one maximal CUDA-rSVD SIMPLS fit to supply every prefix.

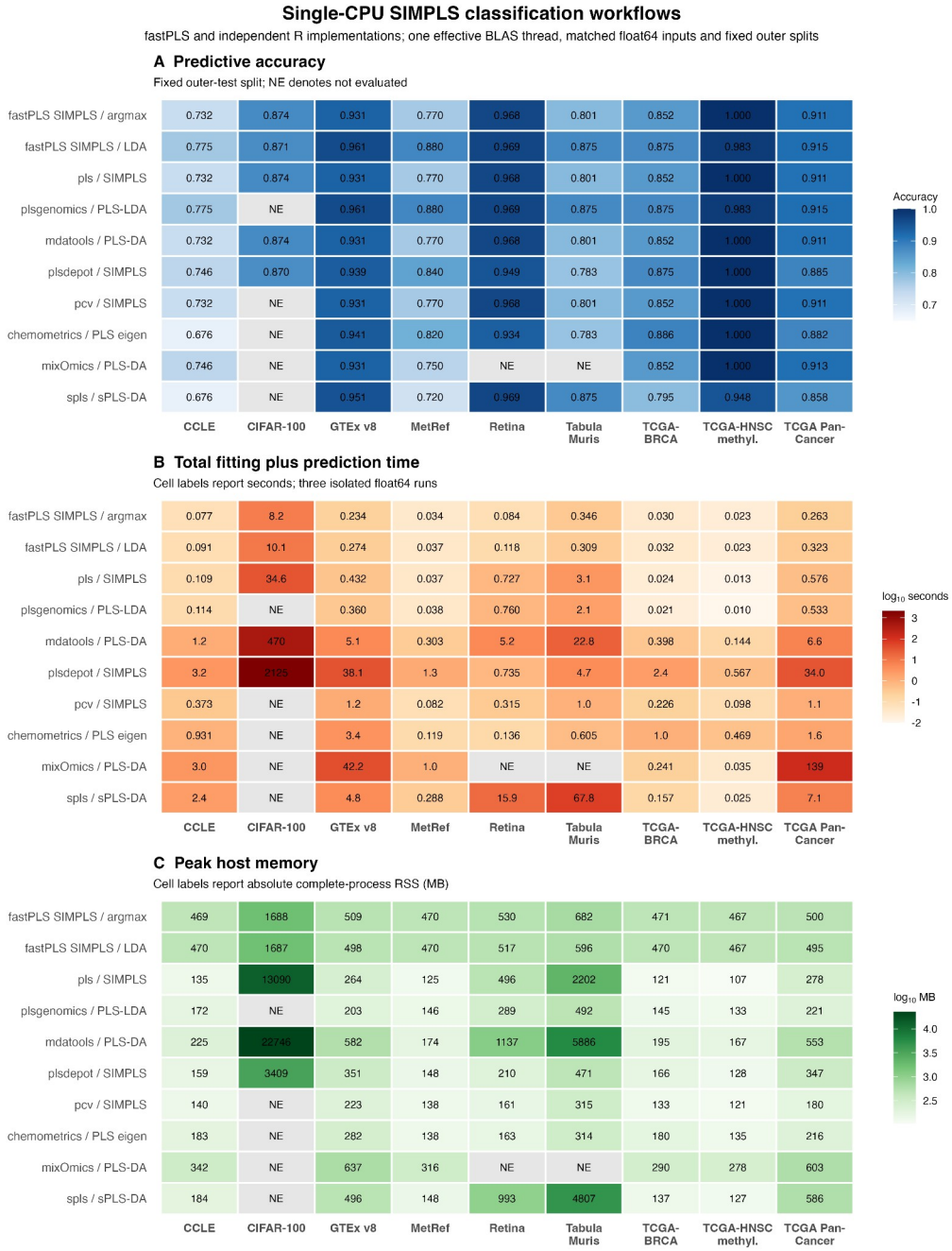

Figure 2. Archived float64 single-CPU SIMPLS classification workflows. Panels report fixed-split accuracy, total fitting-plus-prediction time, and absolute complete-process peak RSS from three isolated runs; NE denotes not evaluated. Argmax rows provide the closest estimator comparison; LDA and other PLS-DA rows compare complete workflows with different heads or retained outputs.

3.3 Internal acceleration and low-rank solver choice

Every automatic route completed execution, but numerical qualification was not uniform: agreement fell outside the predefined tolerances in parts of the retained-component, class-count, cross-covariance-rank, and CUDA response-dimension sweeps. Runtime and memory summaries therefore exclude those discordant routes rather than treating successful execution as numerical validation.

The controlled study completed all 486 CPU/CUDA runs. Automatic rSVD met tolerance at all sample-size, predictor-size, requested-prefix, and cross-covariance-size points, but not throughout the retained-component, class-count, rank, or CUDA response-dimension sweeps; 73 of 276 automatic-route runs were excluded from validated speed claims. Among routes that met tolerance, CUDA first exceeded CPU rSVD at n = 5,000 and p = 2,000 and was 3.04-5.97-fold faster across the 2-64 MiB cross-covariance sweep. CPU implicit products reduced incremental RSS by 29.7-47.5%, but were slower below 32 MiB and approximately time-neutral at 32-64 MiB. A 56-run Metal diagnostic completed without execution failure, but all 20 Metal rSVD routes were numerically discordant and excluded from validated performance claims (Supplementary Tables S7c-S7d; Figure S1).

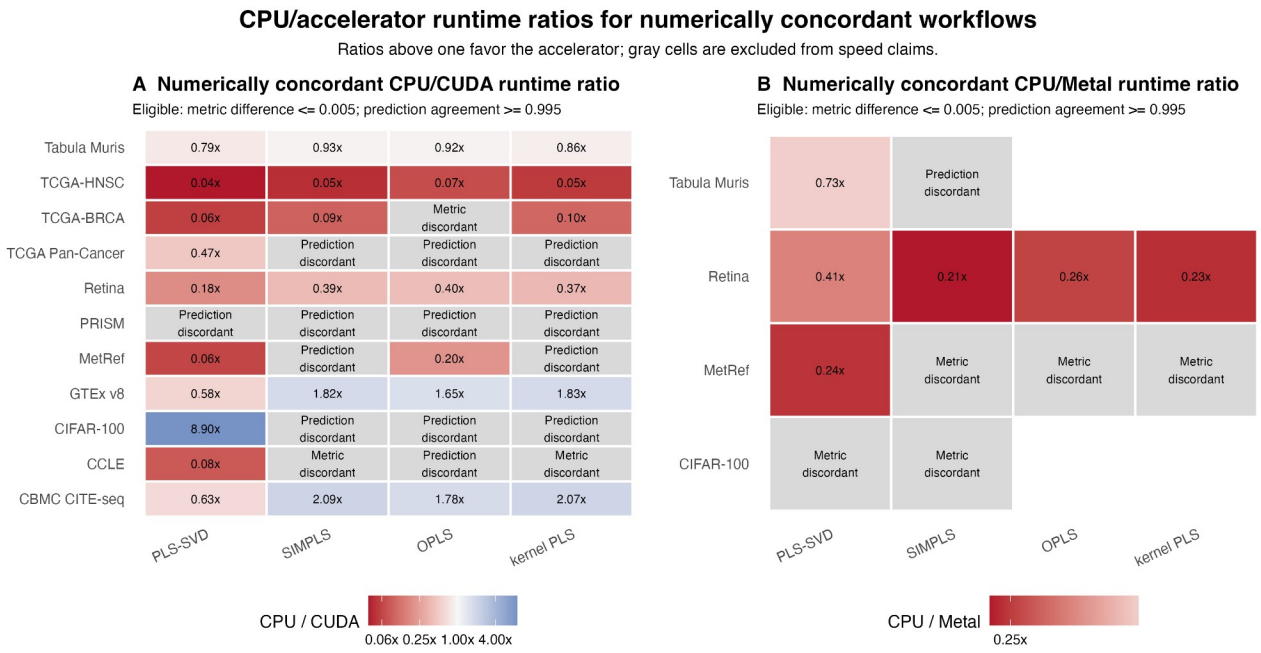

Figure 3. Archived CPU/accelerator runtime ratios for numerically concordant workflows. Ratios are CPU time divided by accelerator time, so values above one favor CUDA or Metal and values below one indicate accelerator slowdown. Cells are colored only when the absolute paired predictive-metric difference is at most 0.005 and prediction agreement is at least 0.995. Gray cells are retained as metric- or prediction-discordant and excluded from acceleration claims.

The computational benefit of hardware acceleration depended on both the execution route and workload dimensions. Among the 44 CPU/CUDA pairs, 28 met the predefined numerical-concordance criteria and seven favored CUDA, including an 8.90-fold PLS-SVD runtime ratio on CIFAR-100 (Figure 3). Six of the 12 CPU/Metal pairs met the same criteria, but none favored Metal. Discordant routes remain visible as gray cells and were not converted into acceleration claims. A separate frozen-release SIMPLS-rSVD comparison is reported in Supplementary Figure S4 and Table S11. CPU IRLBA remains the deterministic numerical reference, and float32 did not uniformly improve runtime, process memory, or numerical agreement (Supplementary Tables S8-S9).

3.4 Multivariate NMR prediction

Training-only selection by the one-standard-error rule retained five PLS-SVD components and 50 SIMPLS components, both interior to their evaluated grids. At these settings, PLS-SVD achieved RMSD 0.001043 and Q^2 0.98916 across CPU IRLBA, CPU rSVD, and CUDA rSVD. SIMPLS achieved RMSD 0.00075608, 0.00075595, and 0.00075606 and Q^2 0.994299, 0.994301, and 0.994299, respectively. Figure 4 displays held-out sample AMI-0030-9 (index 38), selected by the predefined rule of closest per-spectrum RMSD to the median under 50-component SIMPLS CUDA rSVD rather than by visual concordance.

At five PLS-SVD components, CPU IRLBA, CPU rSVD, and CUDA rSVD required 262.86, 4.57, and 0.422 s, respectively. At 50 SIMPLS components, the corresponding times were 692.21, 152.09, and 3.91 s. Every approximate row met the predefined numerical tolerances. The separate 100-component implementation comparison is reported in the Supplement because it addresses a different question from family-specific predictive selection.

The deposited 165-component PLS-SVD/IRLBA workflow required 447.6 s and achieved RMSD 0.000710. It is shown as scientific context for the earlier Nature Communications analysis, not as an implementation-only contrast, because component count, workflow, and hardware differ. Public IKPLS 6.1.2 was also tested in float32. Its one-component fit required 56.58 s and 4.18 GiB incremental RSS but emitted a near-zero-weight warning and returned the training-response mean (RMSD 0.002113), so it did not provide a valid predictive component. The five-component run exceeded the 10-GiB guard, and the retained coefficient path alone would require 68.66 GiB at the selected 50-component SIMPLS setting. IKPLS was therefore infeasible for the scientifically relevant NMR model on the evaluated hardware.

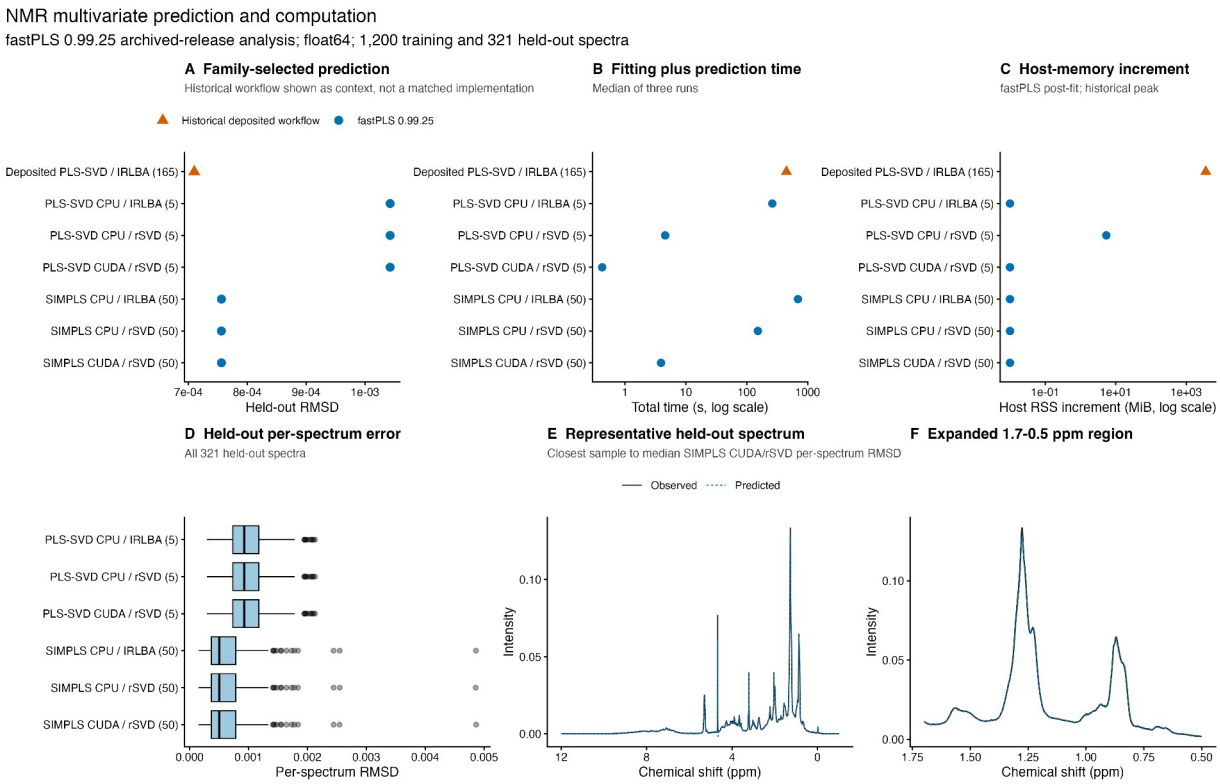

Figure 4. Archived-release NMR prediction and computation. Panels A-C compare family-selected held-out RMSD, total time, and host-memory measurements; the deposited 165-component PLS-SVD/IRLBA workflow is historical scientific context, not a matched implementation. Panel D shows per-spectrum RMSD for all 321 held-out spectra. Panels E-F overlay observed and predicted intensities for AMI-0030-9 (index 38), the sample closest to the median SIMPLS CUDA/rSVD error, over the full response range and 1.7-0.5 ppm. Predictor water-region columns (4.6-4.8 ppm) were zeroed in training and test predictors; all 28,355 response coordinates remained in every metric. rSVD used oversampling 20, two power iterations, and seed 123.

3.5 Foundation-model embedding feasibility

Archived fastPLS 0.99.25 completed the million-row SIMPLS fit and blocked prediction of all 281,167 held-out embeddings (Figure 5). Public IKPLS 6.1.2 also completed float32 cross-product fits at 100, 200, 500, and 1,000 components on one CPU thread. Its top-1/top-5 accuracy increased from 62.91%/86.40% at 100 components to 79.99%/95.61% at 1,000 components, while model fitting plus blocked prediction increased from 52.67 to 197.06 s and peak process RSS from 8.24 to 11.79 GiB. These IKPLS accuracies closely followed the corresponding

fastPLS argmax trajectory, but the timing comparison is not estimator matched: IKPLS refitted each requested component count, whereas one maximal fastPLS CUDA-rSVD SIMPLS fit supplied all prefixes. The results demonstrate downstream matrix-processing feasibility; they do not establish representation-level reproducibility, biomedical utility, or an optimized ImageNet classifier, and the 1,000-component endpoint remains a boundary stress point. Full values and provenance limitations are reported in Supplementary Sections S15.2 and S18.

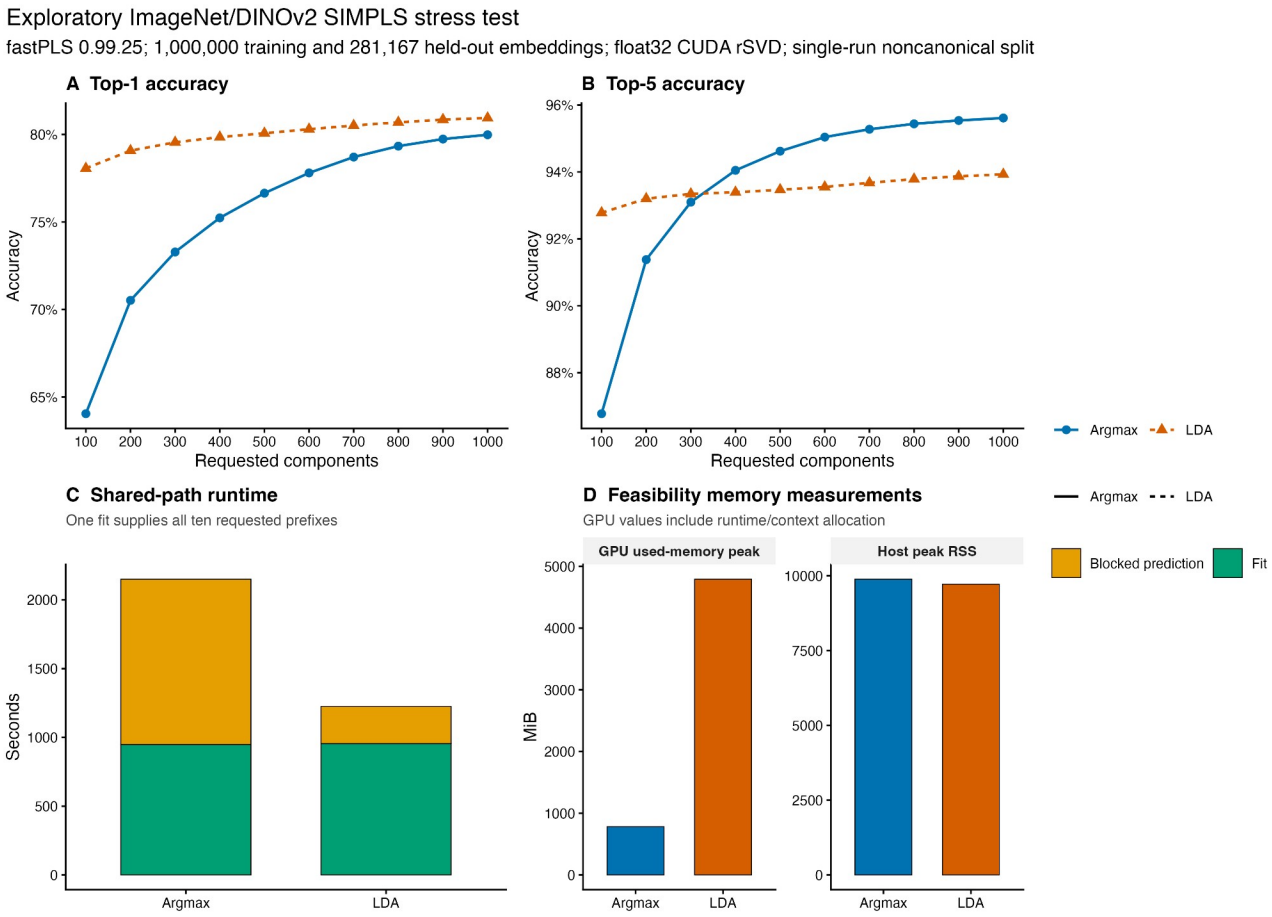

Figure 5. Historical, partially reproducible ImageNet/DINOv2 downstream SIMPLS feasibility experiment. fastPLS 0.99.25 reran label-aware float32 CUDA-rSVD fitting and blocked prediction on 1,000,000 training and 281,167 held-out stored embeddings. Panels report single-run top-1 and top-5 accuracy for argmax and LDA, shared-path fitting and prediction time, and complete-process host and device-memory measurements across 100-1,000 component prefixes. rSVD used oversampling 20, two power iterations, and seed 123. The split was noncanonical; checkpoint, pooling, extraction, and complete image-to-row provenance were unavailable. The figure is an engineering stress test, not biomedical or representation-quality validation, and 1,000 components is a boundary stress point rather than an optimum.

4. Discussion

The principal gain arises from reusing sequential SIMPLS quantities, updating coefficients and fitted values incrementally, and predicting from compact latent factors rather than from retained dense paths. The benefit increases when the response dimension, test set, or number of requested component prefixes is large: compact prediction reduced incremental RSS by up to 77.7%, but offered little benefit when those outputs were intrinsically small. PLS-family and hardware choices remain workload dependent. On one CPU, PLS-SVD was as fast as or faster than SIMPLS across the five matched synthetic matrices (SIMPLS/PLS-SVD time ratio 1.00-3.84), as expected for a one-shot decomposition. On the qualified CUDA cases, SIMPLS approached or marginally exceeded PLS-SVD speed (ratio 0.92-0.98), while retaining sequential orthogonalization and support for component counts not restricted by response rank. Family selection should nevertheless use training-only predictive validation because PLS-SVD and SIMPLS are different estimators. In the broader R-package panel, fastPLS had the lowest observed total time on seven of nine classification datasets, although the matched minimal-output comparison with pls::simpls.fit showed smaller, dataset-dependent differences. IKPLS was faster in the single-thread cross-language experiments and reproduced the ImageNet argmax trajectory closely, but its retained coefficient path made the selected NMR model infeasible even in float32. This contrast emphasizes that performance depends on matrix shape and storage policy: fastPLS contributes an R-native de Jong SIMPLS workflow, compact multivariate-response prediction, nested validation, multiple PLS families, and route diagnostics rather than universal superiority over every PLS implementation.

Optional storage routes and hardware acceleration provide secondary, workload-specific gains. Implicit cross-covariance products are useful when explicitly storing the predictor-response

cross-product or deflated intermediates is the memory bottleneck. In the controlled CPU study they reduced incremental RSS by 29.7-47.5%, but were slower below 32 MiB and approximately time-neutral at 32-64 MiB. CUDA first surpassed CPU rSVD at n = 5,000 in the sample-size sweep and p = 2,000 in the predictor-size sweep. These hardware-specific crossovers depend on transfer, context creation, synchronization, aspect ratio, and device memory. Metal was disadvantaged by host-assisted stages and dispatch overhead; its discordant rSVD routes were excluded from valid accelerator comparisons.

Solver choice separates exploratory acceleration from confirmatory analysis. rSVD with oversampling 20 and two power iterations is the primary approximate route, but scientifically consequential analyses should inspect case diagnostics and compare several predefined seeds. Material seed variation, diagnostic failure, ill-conditioning, or a requirement for deterministic reproducibility indicates CPU IRLBA. Boundary component counts remain best within the evaluated grid rather than optima. Nonlinear kernel PLS requires an n-by-n Gram matrix; the matrix alone occupies approximately 0.75, 4.66, and 18.63 GiB in float64 at n = 10,000, 25,000, and 50,000, before copies and workspaces. Unsupported combinations stop explicitly, whereas experimental or unqualified approximate routes warn and record diagnostics. Supplementary Table S6a summarizes these decisions.

5. Conclusions

fastPLS reduces avoidable computation and storage along the sequential SIMPLS component path while preserving the deterministic de Jong estimator within the predefined numerical tolerances. OPLS, kernel PLS, approximate low-rank solvers, and accelerator backends extend this core contribution under route-specific validation; deterministic float64 CPU SIMPLS remains the confirmatory reference.

Ethics statement

This software study used public, simulated, or previously collected de-identified data; source-study ethics are reported in the cited publications.

CRediT authorship contribution statement

Dupe Ojo: Investigation, validation, data curation, visualization, writing - review and editing. Alessia Vignoli: Investigation, validation, data curation, visualization, writing - review and editing. Stefano Cacciatore: Conceptualization, methodology, software, supervision, project administration, writing - original draft, writing - review and editing. Leonardo Tenori: Conceptualization, methodology, supervision, writing - review and editing.

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors. Computing resources are acknowledged separately below.

Acknowledgements

The authors thank the University of Cape Town's ICTS High Performance Computing team for providing a high-performance computing facility for this study (https://ucthpc.uct.ac.za/).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

Code and benchmark outputs are available at https://github.com/tkcaccia/fastPLS; reusable components are at https://github.com/tkcaccia/kodama-cpp. Central current-version evidence was generated with fastPLS 0.99.25, Git commit

7887401b09e25f54a546a253c255741cb1ab48e5, from source archive fastPLS_0.99.25.tar.gz (SHA-256 604f74d72f5e9540f7378efd37f2ca6ed87ccb9e73b912211331a8cd97233481). The IKPLS large-case float32 scripts and archived summary are in benchmark/ikpls_cross_language/. The ImageNet PLS fitting and prediction stages were rerun from that archive, but the older DINOv2 feature-extraction checkpoint, pooling rule, script, and image-to-row mapping are not fully recoverable; those results are therefore historical and partially reproducible. The deposited 165-component NMR workflow remains a separately identified historical context row.

References

[1] Wold S, Sjostrom M, Eriksson L. PLS-regression: a basic tool of chemometrics. Chemom Intell Lab Syst. 2001;58:109-130. https://doi.org/10.1016/S0169-7439(01)00155-1.

[2] Geladi P, Kowalski BR. Partial least-squares regression: a tutorial. Anal Chim Acta. 1986;185:1-17. https://doi.org/10.1016/0003-2670(86)80028-9.

[3] Barker M, Rayens W. Partial least squares for discrimination. J Chemom. 2003;17:166-173. https://doi.org/10.1002/cem.785.

[4] Bertini I, Cacciatore S, Jensen BV, et al. Metabolomic NMR fingerprinting to identify and predict survival of patients with metastatic colorectal cancer. Cancer Res. 2012;72:356-364. https://doi.org/10.1158/0008-5472.CAN-11-1543.

[5] Cacciatore S, Wium M, Licari C, et al. Inflammatory metabolic profile of South African patients with prostate cancer. Cancer Metab. 2021;9:29. https://doi.org/10.1186/s40170-021-00265-6.

[6] Elebo N, et al. Serum metabolomic and lipoprotein profiling of pancreatic ductal adenocarcinoma patients of African ancestry. Metabolites. 2021;11:663. https://doi.org/10.3390/metabo11100663.

[7] Vignoli A, Cacciatore S, Tenori L. Deriving three one-dimensional NMR spectra from a single experiment through machine learning. Nat Commun. 2025;16:10159. https://doi.org/10.1038/s41467-025-65294-x.

[8] Cacciatore S, Tenori L, Luchinat C, Bennett PR, MacIntyre DA. KODAMA: an R package for knowledge discovery and data mining. Bioinformatics. 2017;33:621-623. https://doi.org/10.1093/bioinformatics/btw705.

[9] Cacciatore S, Luchinat C, Tenori L. Knowledge discovery by accuracy maximization. Proc Natl Acad Sci USA. 2014;111:5117-5122. https://doi.org/10.1073/pnas.1220873111.

[10] Wegelin JA. A survey of partial least squares methods, with emphasis on the two-block case. University of Washington; 2000.

[11] de Jong S. SIMPLS: an alternative approach to partial least squares regression. Chemom Intell Lab Syst. 1993;18:251-263. https://doi.org/10.1016/0169-7439(93)85002-X.

[12] Trygg J, Wold S. Orthogonal projections to latent structures (O-PLS). J Chemom. 2002;16:119-128. https://doi.org/10.1002/cem.695.

[13] Lindgren F, Geladi P, Wold S. The kernel algorithm for PLS. J Chemom. 1993;7:45-59. https://doi.org/10.1002/cem.1180070104.

[14] Rosipal R, Trejo LJ. Kernel partial least squares regression in reproducing kernel Hilbert space. J Mach Learn Res. 2001;2:97-123.

[15] Baglama J, Reichel L. Augmented implicitly restarted Lanczos bidiagonalization methods. SIAM J Sci Comput. 2005;27:19-42. https://doi.org/10.1137/04060593X.

[16] Halko N, Martinsson PG, Tropp JA. Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. SIAM Rev. 2011;53:217-288. https://doi.org/10.1137/090771806.

[17] Fisher RA. The use of multiple measurements in taxonomic problems. Ann Eugen. 1936;7:179-188.

[18] McLachlan GJ. Discriminant Analysis and Statistical Pattern Recognition. Wiley; 1992.

[19] Mevik BH, Wehrens R. The pls package: principal component and partial least squares regression in R. J Stat Softw. 2007;18:1-23. https://doi.org/10.18637/jss.v018.i02.

[20] Assfalg M, et al. Evidence of different metabolic phenotypes in humans. Proc Natl Acad Sci USA. 2008;105:1420-1424. https://doi.org/10.1073/pnas.0705685105.

[21] Stoeckius M, et al. Simultaneous epitope and transcriptome measurement in single cells. Nat Methods. 2017;14:865-868. https://doi.org/10.1038/nmeth.4380.

[22] GTEx Consortium. The GTEx Consortium atlas of genetic regulatory effects across human tissues. Science. 2020;369:1318-1330. https://doi.org/10.1126/science.aaz1776.

[23] Ghandi M, et al. Next-generation characterization of the Cancer Cell Line Encyclopedia. Nature. 2019;569:503-508. https://doi.org/10.1038/s41586-019-1186-3.

[24] Hoadley KA, et al. Cell-of-origin patterns dominate the molecular classification of 10,000 tumors from 33 types of cancer. Cell. 2018;173:291-304.e6. https://doi.org/10.1016/j.cell.2018.03.022.

[25] Macosko EZ, Basu A, Satija R, et al. Highly parallel genome-wide expression profiling of individual cells using nanoliter droplets. Cell. 2015;161:1202-1214. https://doi.org/10.1016/j.cell.2015.05.002.

[26] Tabula Muris Consortium. Single-cell transcriptomics of 20 mouse organs creates a Tabula Muris. Nature. 2018;562:367-372. https://doi.org/10.1038/s41586-018-0590-4.

[27] Corsello SM, et al. Discovering the anticancer potential of non-oncology drugs by systematic viability profiling. Nat Cancer. 2020;1:235-248. https://doi.org/10.1038/s43018-019-0018-6.

[28] Deng J, et al. ImageNet: a large-scale hierarchical image database. Proc IEEE CVPR. 2009:248-255. https://doi.org/10.1109/CVPR.2009.5206848.

[29] Oquab M, Darcet T, Moutakanni T, Vo HV, Szafraniec M, Khalidov V, et al. DINOv2: learning robust visual features without supervision. Trans Mach Learn Res. 2024. https://openreview.net/forum?id=a68SUt6zFt; arXiv:2304.07193.

[30] Krizhevsky A, Hinton G. Learning multiple layers of features from tiny images. Technical report. University of Toronto; 2009. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

[31] Chen RJ, Ding T, Lu MY, et al. Towards a general-purpose foundation model for computational pathology. Nat Med. 2024;30:850-862. https://doi.org/10.1038/s41591-024-02857-3.

[32] Xu H, Usuyama N, Bagga J, et al. A whole-slide foundation model for digital pathology from real-world data. Nature. 2024;630:181-188. https://doi.org/10.1038/s41586-024-07441-w.

[33] Dayal BS, MacGregor JF. Improved PLS algorithms. J Chemom. 1997;11:73-85. https://doi.org/10.1002/(SICI)1099-128X(199701)11:1<73::AID-CEM435>3.0.CO;2-#.

[34] Engstrom OCG, Dreier ES, Jespersen BM, Pedersen KS. IKPLS: Improved Kernel Partial Least Squares and fast cross-validation algorithms for Python with CPU and GPU implementations using NumPy and JAX. J Open Source Softw. 2024;9:6533. https://doi.org/10.21105/joss.06533.

[35] Engstrom OCG. Shortcutting cross-validation: efficiently deriving column-wise centred and scaled training-set X-transpose-X and X-transpose-Y without full recomputation. arXiv. 2024. https://doi.org/10.48550/arXiv.2401.13185.

[36] Johnson J, Douze M, Jegou H. Billion-scale similarity search with GPUs. IEEE Trans Big Data. 2021;7:535-547. https://doi.org/10.1109/TBDATA.2019.2921572.